

TP5 — Experimentos extra

Borrador para discusion: ¿cuales integramos a la presentacion principal?

De que se trata este anexo

Cuatro experimentos **adicionales** (numpy puro, reusando el VAE ya gradchequeado del EJ2). No estan en la presentacion principal; la idea es verlos y decidir cuales valen la pena.

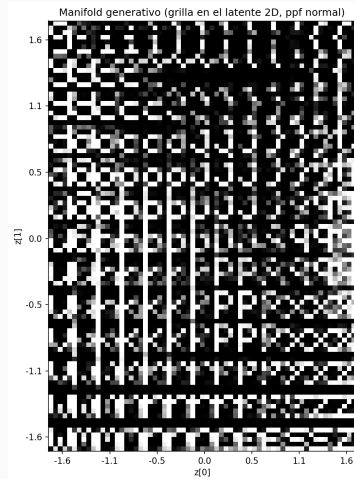
- **A — VAE sobre el dataset del EJ1 (las letras).** Puente AE→VAE sobre los *mismos* datos: ¿que aporta el VAE?
- **B — Unidades activas / posterior collapse.** *Por que* satura el barrido de dimension latente (16–32).
- **D — Generalizacion (split train/test).** ¿El VAE de emojis memoriza o generaliza? (disciplina del TP3, hoy ausente en EJ2).
- **E — MSE vs BCE.** El clasico “MSE borroso, BCE nitido”.

Todo reproducible: `python3 extra/{vae_letters,active_units,generalization,recon_loss}.py`.

A — VAE sobre el dataset del EJ1 (letras)

El VAE sobre las 32 letras (puente AE $\rightarrow$ VAE)

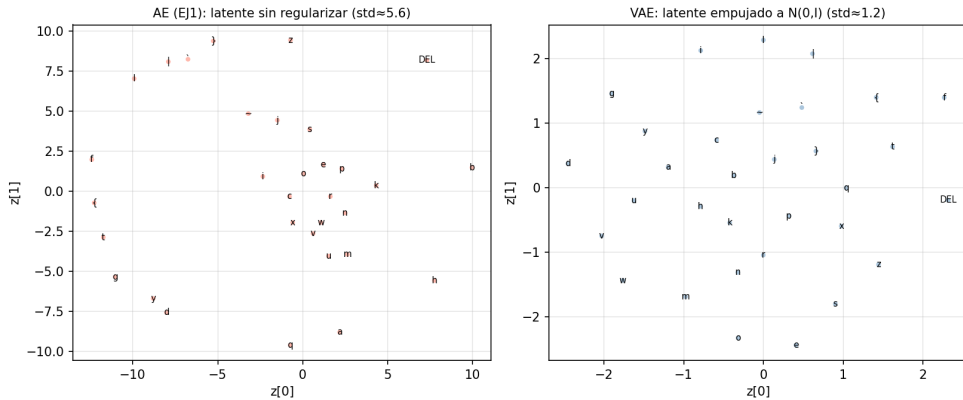
- Entrenamos el **VAE del EJ2** sobre las **32 letras** del EJ1 (input 35, latente 2D, BCE, Adam).
- Reconstruye casi perfecto: **31/32** exactas (max = 1 px); el AE basico memoriza 32/32.
- Pero ademas el VAE arma un **manifold continuo**: cada punto del plano latente decodifica una letra plausible.



Manifold 2D: grilla de z decodificada (se ven P,o,d,k,z...).

AE vs VAE: ¿que compra el termino KL?

Espacio latente 2D sobre las 32 letras: AE vs VAE



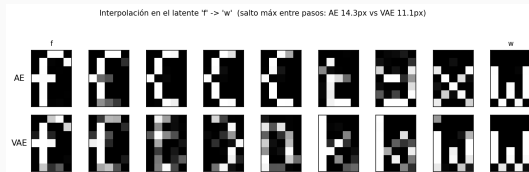
Mismos datos, latente 2D. El **AE** desparrama los codigos sin estructura (std $\approx 5,6$, rango ± 13); el **VAE** los empuja a $\mathcal{N}(0, I)$ (std $\approx 1,2$, centrado en 0). Por eso *muestrear del prior* funciona en el VAE y no en el AE.

AE vs VAE: muestreo del prior e interpolacion



Mismos $z \sim \mathcal{N}(0, I)$ decodificados. Dist. media a letra real:

VAE 1.6 px vs AE 2.2 px.



Interpolacion $f \rightarrow w$. Salto máximo entre pasos: **VAE 11.1 px**
vs AE 14.3 px (mas gradual).

Con solo 32 puntos el efecto es *modesto* pero consistente: el latente del VAE es mas usable como espacio generativo. En emojis/MNIST (mas datos) la diferencia es mayor.

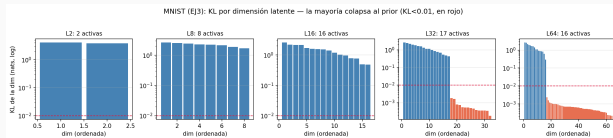
**B — Unidades activas
(posterior collapse)**

¿Por que satura el latente? Unidades activas

- KL *por dimension*:

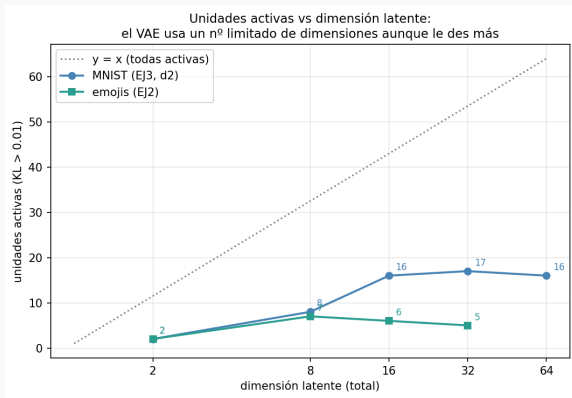
$$KL_j = \mathbb{E}_x \left[-\frac{1}{2} (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \right]$$

- Si la dim j **colapsa** al prior
 $\Rightarrow KL_j \approx 0$ (no transporta informacion).
- “Activa” = $KL_j > 0,01$.
- En MNIST, con latente 32 o 64 la **mayoria de las dims colapsan** (rojo).



MNIST (EJ3): KL por dimension (ordenada). En rojo, las dims colapsadas (KL < 0,01).

El VAE usa un numero limitado de dimensiones



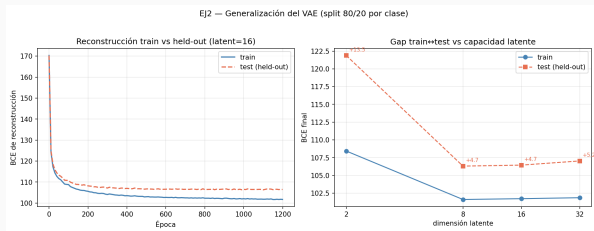
MNIST (EJ3, arquitectura d2):

latente	activas	KL total
2	2/2	7,9
8	8/8	17,8
16	16/16	22,5
32	17/32	22,5
64	16/64	22,5

El mecanismo de la saturacion: aunque demos 32 o 64 dimensiones, el VAE solo activa ~16–17; el KL total se clava en $\approx 22,5$. Por eso la calidad no mejora pasando de 16–32.

D — Generalizacion (train/test)

¿El VAE de emojis memoriza o generaliza?



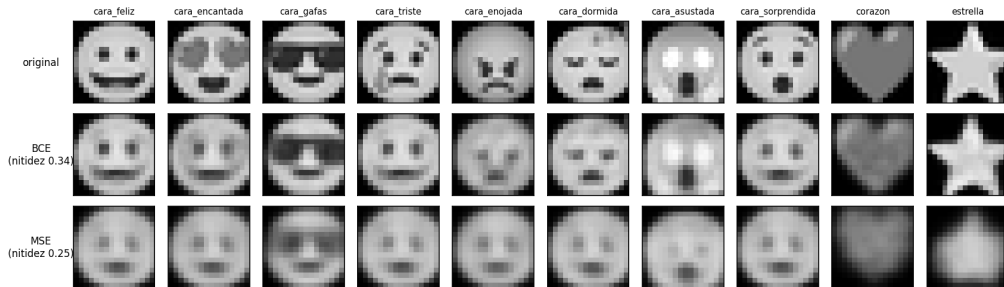
- Split **80/20** estratificado por clase (held-out de variantes nunca vistas); disciplina del TP3, ausente en EJ2.
- El test **sigue al train** (no sube): **generaliza**, no memoriza. Gap chico ($\sim +5$ BCE).
- El gap **no crece** con la dim latente (8/16/32 dan $\sim +5$; latente 2 *underfitea*, +13).
- **Conecta con B**: el posterior collapse limita la capacidad *efectiva* ($\sim 5-7$ dims activas), así que mas latente *no* agrega sobreajuste.



E — MSE vs BCE

Perdida de reconstruccion: MSE vs BCE

VAE emojis (latent=16): BCE más nítida, MSE más borrosa



Dos VAE identicos salvo la `recon_loss` (misma salida logistica, latente 16, 600 ep). **BCE** es mas nitida (nitidez 0,34 vs 0,25) y ademas reconstruye mejor *aun medido en MSE* (1,4 vs 5,2): MSE+sigmoide tiene gradiente que se **desvanece** cerca de 0/1, asi que entrena peor a igual presupuesto.

Resumen

- **B (posterior collapse)** — el mas fuerte: explica *cuantitativamente* por que el latente satura (solo ~ 16 dims activas de 64). **Recomendado integrar.**
- **A (VAE sobre letras)** — buen puente narrativo AE $\rightarrow$ VAE sobre datos ya conocidos; el scatter AE vs VAE es muy claro.
- **E (MSE vs BCE)** — barato y visual; justifica la eleccion de BCE de la principal.
- **D (train/test)** — aporta rigor (el VAE *generaliza*; gap acotado por el posterior collapse de B, no crece con el latente).

Todos los datos crudos quedan en `extra/results/` (*.csv, *.npz) para regraficar sin re-entrenar.